



MSA UNIVERSITY
جامعة أكتوبر للعلوم الحديثة والآداب



October University for Modern Science and Arts

Faculty of Computer Science

Machine Learning Project Report

Course Name: Machine Learning

Course Code: CS363

Instructor: Assoc Prof. Nermin Abd El-Wahab, Dr. Ebtsam El-Hussany

Teaching Assistant: Eng. Omnia Fawzy, Eng. Youssef Araby, Eng. Mazen Ashraf,
Eng. Ahmed Yasser

Team Members:

Mazen Ismail – 246063

Marwan Ismail – 246069

Omar Khalifa – 247923

Mohamed Hany – 246631

Malak Foudh – 245229

Submission Date: May 16, 2026

Contents

1	Introduction	2
2	Literature Review	3
2.1	Overview of Related Work	3
2.2	Summary of Key Papers	4
2.3	Gap Analysis	6
3	Dataset Description	6
4	Data Preprocessing and Feature Engineering	8
5	Implemented Models	12
6	Evaluation Strategy	16
7	Results and Comparative Analysis	17
8	Error Analysis and Critical Reflection	18
9	Conclusion and Future Work	20
10	Teamwork and Task Delegation	21
11	Appendix	21

1 Introduction

CF-Analyzer is a personalized problem recommendation system using KNN and LightGBM, trained on over 800,000 submissions from 2,877 Codeforces users.

Motivation

- Beginner users on Codeforces struggle to find an effective strategy to improve their skill level, due to the massive number of available problems and the variety of topics they cover.
- Users tend to solve easy problems near their current skill level, leading to a performance plateau.
- There is no built-in metric for users to evaluate their skill level relative to peers.
- Improvement strategies for advanced users are not applicable to beginners due to the significant skill gap; weakness profiles must be personalized.

Objective

Provide an ML pipeline that tailors a personalized study plan for each user by analyzing their strengths, weaknesses, and improvement metrics relative to similar peers.

Dataset

The dataset was self-collected from the Codeforces API, then preprocessed and filtered using custom Python scripts.

Models Used

- KNN — finding the 50 nearest neighbors in an 82-dimensional feature space
- LightGBM Regressor — problem success scoring and attempts estimation

Alternative Models Evaluated

- XGBoost (requires balanced data; discarded due to class imbalance)
- Linear Regression (weaker nonlinear capture; used as baseline)
- Logistic Regression (failed due to skewness of the target distribution)

Summary of Achieved Results

The system delivers an end-to-end ML-powered competitive programming coach.

Dataset & KNN

- A dataset of 2,877 active Codeforces users was collected and preprocessed into an 82-dimensional feature space spanning 20 algorithmic topics.
- Leave-one-out KNN evaluation ($K=50$, Euclidean distance) yielded:
 - Weakness Recall: 85.78% (target: $>70\%$)
 - Weakness Jaccard: 64.34% (target: $>50\%$)
 - Tag MSE: 0.0075 (target: <0.05)
 - Spearman Correlation: 0.634

ML Models

- The *Success Model* (LightGBM) achieves AUC-ROC of 0.78 and F1-score of 0.72, used to rank problems by difficulty fit.
- The *Attempts Estimator* (LightGBM) achieves MAE of ± 0.88 attempts, outperforming Linear Regression (MAE ≈ 1.1) and Logistic Regression (MAE ≈ 1.0).
- Both models were trained on $\approx 500,000$ and $\approx 384,000$ rows respectively from the preprocessed dataset.

2 Literature Review

2.1 Overview of Related Work

CF-Analyzer sits at the intersection of competitive programming analytics, ML-based performance prediction, and personalised problem recommendation. Prior work has approached these sub-problems using classical ML classifiers (Logistic Regression, Naïve Bayes, SVM, Random Forest), deep learning architectures (MLP, CNN, LSTM, and transformer-based models), and content-based filtering — all applied to online-judge data such as submission histories, rating trajectories, and problem tags. The three papers reviewed below are the most directly relevant to our system’s core pipeline: problem attribute prediction, contest performance forecasting, and automated problem recommendation.

2.2 Summary of Key Papers

Paper 1 — Kim et al. (2024): Problem-Solving Guide (PSG)

Reference: Kim, J., Cho, E., & Na, D. *Problem-Solving Guide (PSG): Predicting the Algorithm Tags and Difficulty for Competitive Programming Problems*. AAAI Conference on Artificial Intelligence (AAAI 2024), *Proceedings of Machine Learning Research*, Vol. 257, pp. 48–56. [1]

Problem addressed. The paper frames the prediction of a Codeforces problem’s algorithmic category (tag) and difficulty rating as a unified multi-task learning problem — multi-label classification for tags, and ordinal classification for difficulty.

Dataset. A corpus of 7,976 Codeforces problems with their problem statements, tags, and difficulty ratings.

Models implemented. A BigBird-based multi-task model (PSG), outperforming earlier single-task CNN-based approaches on both prediction tasks.

Performance. PSG achieves state-of-the-art results on both the tag classification and difficulty prediction tasks, surpassing all single-task baselines.

Limitations. The model operates purely on problem text and has no notion of user behaviour; it cannot personalise recommendations based on individual skill profiles.

Relation to this project. Our problem recommendation pipeline relies on accurately labelled tags and difficulty ratings when scoring candidate problems for a target user. PSG validates that these attributes are machine-learnable and motivates treating tag coverage as a first-class feature in our 29-dimensional LightGBM input vector.

Paper 2 — Rahman et al. (2023): Predicting Contest Performance with Deep Learning

Reference: Rahman, M. M., Chandra Das, B., Biswas, A. A., & Anwar, M. M. *Predicting Participants’ Performance in Programming Contests Using Deep Learning Techniques*. *Hybrid Intelligent Systems (HIS 2022)*, Lecture Notes in Networks and Systems, Vol. 647, Springer. [2]

Problem addressed. Predicting a contestant’s upcoming contest rank and post-contest rating using their Codeforces history.

Dataset. Submission histories, contest ranks, ratings, and verdicts for 100 Codeforces users, treating each contest as a time-step with four feature types.

Models implemented. Multiple ML baselines (Logistic Regression, SVM, Random

Forest) and deep learning models (MLP, LSTM); deep learning outperformed all baselines across every metric.

Performance. Deep learning models achieve the lowest prediction error on both rank and rating forecasting tasks.

Limitations. The 100-user sample is too small for robust generalisation. Only four per-contest features are used, ignoring tag-level granularity and submission-level behavioural signals.

Relation to this project. This paper motivates using nonlinear models over linear ones for user-level performance prediction — a finding confirmed by our own comparison experiments (LightGBM vs. Linear Regression, Section 7). Our dataset of 2,877 users with an 82-dimensional feature space directly addresses the small-sample and shallow-feature limitations identified here.

Paper 3 — Sudha et al. (2018): CNN-Based Problem Classification and Recommendation

Reference: Sudha, S., Arun Kumar, A., Muthu Nagappan, M., & Suresh, R. *Classification and Recommendation of Competitive Programming Problems Using CNN*. *ICIIT 2017*, Communications in Computer and Information Science, Vol. 808, Springer. [3]

Problem addressed. Automatically identifying the algorithmic technique required by a problem from its C++ solution code, and using that classification to recommend similar problems.

Dataset. Competitive programming problems with associated accepted C++ solutions, labelled by algorithmic technique.

Models implemented. A CNN applied directly to tokenised source-code representations, one of the earliest works combining automated problem classification with a recommendation pipeline in the competitive programming domain.

Performance. The CNN-based classifier achieves higher accuracy than traditional text-feature baselines for technique identification.

Limitations. Requires access to accepted solution code at inference time, which is unavailable when recommending *unsolved* problems. Recommendations are purely content-based and offer no personalisation relative to a user’s individual skill profile.

Relation to this project. While Sudha et al. route recommendations through source-code similarity, CF-Analyzer achieves tag-level personalisation entirely from submission metadata — no solution code is needed. This makes our approach applicable at recommendation time, not only retrospectively.

2.3 Gap Analysis

All three papers share a common limitation: they operate on narrow, shallow feature sets and address only one sub-problem at a time. Kim et al. focus exclusively on predicting problem attributes from problem text, with no modelling of user behaviour. Rahman et al. use only four per-contest features and a dataset of 100 users, making generalisation difficult. Sudha et al. classify problems from solution source code rather than user interaction signals, and offer no personalisation based on individual skill levels.

None of these works model the **interplay between tag-specific weakness, problem difficulty gaps, and peer-benchmarked skill standing** — all of which are natively captured in our 82-dimensional KNN feature space and 29-dimensional LightGBM input vector. CF-Analyzer addresses this gap by building an end-to-end ML pipeline on a significantly richer and more holistic feature space, enabling not only problem scoring but also personalised peer retrieval and contextual weakness identification within a single unified system.

3 Dataset Description

Dataset Source

The dataset was self-collected from the public **Codeforces API** (codeforces.com/api), which provides free, unauthenticated access to submission histories and user profiles. No existing Kaggle or UCI dataset was used; the dataset was purpose-built for this system using custom Python collection and preprocessing scripts.

Dataset Characteristics

Scale The raw dataset contains approximately **2 million submissions** from **2,877 active users**, reduced to $\approx 900,000$ cleaned rows after filtering. The tag strength matrix derived from these submissions contains $\approx 57,540$ rows (one per user-tag pair across 20 topics).

Features Each submission record contains 27 columns, including:

- **Categorical:** user handle, problem ID, problem name, tag flags (`tag_dp`, `tag_greedy`, ..., 20 binary columns)
- **Numerical:** problem rating (difficulty), verdict flags (`is_ac`, `is_wa`, `is_tle`, `is_mle`)

User profiles add aggregate numerical features per user: current and peak Codeforces rating, total accepted problems, first-try rate, average attempts to AC, and solved-problem counts broken down by rating band.

Engineered Features (KNN Feature Vector) The primary representation fed to KNN is an **82-dimensional vector** computed per user, structured as follows:

- $4 \times 20 = 80$ dimensions: acceptance rate, difficulty score, specialization score, and volume score for each of the 20 algorithmic tags
- 2 dimensions: rating boost and efficiency score (user-level constants)

Target Variables Two supervised targets are used for the LightGBM models:

- `solve_score` $\in [-1, 1]$: a continuous measure of how cleanly a user solved a problem, defined as `ever_ac - 0.2 × wa_count`, clipped to $[-1, 1]$.
- `attempts`: number of submissions before AC ($= \text{wa_count} + 1$), clipped to $[1, 10]$.

Class / Target Distribution The `solve_score` distribution is heavily right-skewed: the majority of recorded (user, problem) pairs are clean first-try solves (score near 1.0), with a long tail of failed or messy attempts. Similarly, `attempts` is right-skewed with most values at 1 (clean solve) and a long tail up to the clip value of 10.

Exploratory Data Analysis (EDA)

Tag Distribution Across all 2,877 users, the most frequently attempted tags are *implementation*, *greedy*, and *math*, reflecting Codeforces’ problem set composition. Rare tags such as *fft*, *flows*, and *probabilities* have significantly fewer user–problem interactions, producing sparse observations for those columns.

Rating Distribution Problems in the dataset span Codeforces difficulty ratings from 800 to 3500. The bulk of solved problems cluster in the 1000–2000 range, corresponding to the rating range of the majority of collected users. Problems above 2500 are heavily underrepresented.

Tag Strength Scores The engineered `tag_strength` scores ($\in [0, 1]$) are broadly right-skewed across the dataset: most users score below 0.5 in niche or advanced tags (e.g., *fft*, *geometry*) and above 0.5 in fundamental tags (e.g., *implementation*, *greedy*).

Missing Values Users who have never attempted a given tag receive an implicit tag strength of 0 rather than a missing value, by design of the Laplace-smoothed acceptance rate formula. No missing values exist in the final feature matrix used for KNN.

Dataset Challenges and Limitations

- **Target skewness.** Both `solve_score` and `attempts` are heavily right-skewed. The majority of entries are clean solves, which biases naive models toward predicting easy outcomes. LightGBM handles this naturally via tree splits, but it caused Logistic Regression (ordinal proxy) to underperform.
- **Sparse tags.** Tags such as *fft*, *flows*, and *graph matchings* have very few samples per user, making their component scores unreliable for low-activity users. The Laplace smoothing in the acceptance rate formula partially mitigates this.
- **Minimum activity filter.** Users with fewer than 30 accepted submissions were discarded. This improves signal quality but excludes beginners entirely from the dataset, potentially reducing recommendation quality for very new users.
- **Submission cap.** Only the most recent 500 submissions are fetched per target user at runtime. Long-tenured users with thousands of submissions may have their early activity underrepresented.
- **Data collection bias.** The 2,877 dataset users were sampled from active participants in Codeforces rounds. Casual or irregular participants are underrepresented, so the KNN peer group skews toward consistent competitors.
- **No temporal features.** Submission timestamps are not used. A user who was strong in *dp* two years ago but has not practiced since is indistinguishable from a currently active *dp* specialist.

4 Data Preprocessing and Feature Engineering

Data Cleaning

The raw submission data required several cleaning steps before it could be used for training or KNN indexing. All steps are implemented in `ML/preprocessing/submissionsCleaning.py`.

- **Verdict consolidation.** Time Limit Exceeded (`is_tle`) and Memory Limit Exceeded (`is_mle`) flags were merged into `is_wa`, treating all non-AC outcomes uniformly as wrong answers:

`is_wa = is_wa | is_tle | is_mle`

- **Incomplete submission removal.** Rows where both `is_ac` and `is_wa` were 0 (pending or unscored verdicts) were dropped, as they carry no training signal.
- **Null removal.** Rows with a null `problem_name` were dropped to ensure every record has a valid problem identity.
- **Minimum activity filter.** Users with fewer than 30 accepted submissions were removed. Sparse users produce unreliable tag strength estimates and degrade KNN quality.
- **Unrateable problem removal.** Problems with no difficulty rating *and* no tags were dropped, as they contribute no learnable signal to either the feature vector or the LightGBM models.
- **Column selection.** Only the columns needed downstream were retained: `handle`, `problem_id`, `problem_name`, `problem_rating`, `is_ac`, `is_wa`, and all 20 `tag_*` binary columns. This reduced the working set from $\approx 2\text{M}$ to $\approx 900,000$ rows.

No duplicate submission rows were present after applying the minimum activity filter, since each (`handle`, `problem_id`, `verdict`) combination is a distinct event in the Codeforces API response.

Dataset Preprocessing

Binary Tag Encoding Each problem’s Codeforces tags are mapped to 20 binary columns (`tag_dp`, `tag_greedy`, ..., `tag_constructive`). Tags not in the fixed vocabulary of 20 topics are silently discarded. This is equivalent to multi-hot encoding over a fixed tag vocabulary.

Target Variable Construction

- `solve_score` is computed per (`handle`, `problem_id`) pair by aggregating all submissions:

$$\text{solve_score} = \text{ever_ac} - 0.2 \times \text{wa_count}, \quad \text{clipped to } [-1, 1]$$

- `attempts` is computed only on solved problems:

$$\text{attempts} = \text{wa_count} + 1, \quad \text{clipped to } [1, 10]$$

Normalization No global standardization or normalization was applied to the Light-GBM inputs, as gradient boosted trees are invariant to monotonic feature scaling. The KNN feature vector components are each naturally bounded to $[0, 1]$ by construction (see Feature Engineering below), so no additional normalization was required there either.

Feature Engineering

Tag Strength Score (Offline, per user–tag pair) The central engineered feature is `tag_strength` $\in [0, 1]$, computed by `ML/preprocessing/strength.py` for every (user, tag) pair. It combines six component scores with fixed weights:

$$\text{tag_strength} = 0.30 r_{\text{acc}} + 0.30 r_{\text{diff}} + 0.20 r_{\text{boost}} + 0.10 r_{\text{spec}} + 0.075 r_{\text{eff}} + 0.075 r_{\text{vol}}$$

where:

- $r_{\text{acc}} = \frac{\text{ac_count} + 1}{\text{total_attempts} + 2}$ (Laplace-smoothed acceptance rate)
- $r_{\text{diff}} = \frac{\text{avg_rating_solved}}{3500}$ (normalised average difficulty of solved problems)
- $r_{\text{boost}} = \frac{\text{cf_rating} + \text{cf_max_rating}}{2 \times 3500}$ (user-level rating standing)
- $r_{\text{spec}} = \frac{\text{ac_count}}{\text{total_ac}}$ (fraction of all solves belonging to this tag)
- $r_{\text{eff}} = \text{first_try_rate}$ (fraction of problems solved on first submission)
- $r_{\text{vol}} = \frac{\log(1 + \text{ac_count})}{\log(1 + 50)}$ (volume with diminishing returns, normalised at 50 solves)

82-Dimensional KNN Feature Vector `tag_strength` is intentionally *excluded* from the KNN vector. Instead, the four per-tag component scores (r_{acc} , r_{diff} , r_{spec} , r_{vol}) are used directly for all 20 tags, giving $4 \times 20 = 80$ dimensions. The two user-level scalars (r_{boost} , r_{eff}) are appended once, giving **82 dimensions** total. Including them 20 times would inflate their contribution to Euclidean distances by a factor of 20, which was avoided by design.

Peer-Benchmarked Tag Strength (Runtime) At inference time, `ML/inference/TagAnalyzer.py` recomputes tag strengths benchmarked against the 50 retrieved neighbors rather than against the full dataset. This produces scores that reflect

the user's standing within their peer group, making the displayed weaknesses contextually meaningful.

LightGBM Feature Vector (29 dimensions) Both LightGBM models share an identical 29-dimensional input vector per `(user, problem)` pair:

- `rating_diff = problem_rating - cf_rating`
- `rating_diff_sq = rating_diff2`
- `cf_rating, num_solved`
- `mean_tag_strength, tag_strength_min, tag_strength_max, tag_coverage, problem_tag_count`
- 20 binary `tag_*` flags for the problem

Train/Test Split and Reproducibility

Both LightGBM models were trained on an 80/20 train/test split using `random_state = 42` throughout. The split was applied at the `(handle, problem_id)` row level, not at the user level, since the models predict per-problem outcomes rather than per-user outcomes.

All preprocessing steps (target variable computation, tag strength scoring) were computed on the full dataset before splitting, which is appropriate here because these transformations are deterministic functions of the submission records themselves and do not introduce leakage from the test labels. The LightGBM models use early stopping with a held-out 20% validation portion of the training split (i.e., an internal 64/16/20 effective split) to determine the optimal number of trees without overfitting.

The KNN model is non-parametric and requires no train/test split; its quality was assessed via leave-one-out cross-validation on the full dataset of 2,877 users.

Preprocessing Pipeline

The offline preprocessing pipeline executes in the following order:

1. **Raw collection** — Codeforces API → `01_submissions.csv, 02_user_profiles.csv`
2. **Submission cleaning** — `submissionsCleaning.py` → `04_filtered_submissions.csv`
3. **Tag strength computation** — `strength.py` → `06_user_tag_strengths.csv`

4. **Profile enrichment** — merge profiles with strongest/weakest tag columns → `07_enriched_user_profiles.csv`
5. **KNN matrix construction** — pivot `06_user_tag_strengths.csv` into an $2,877 \times 82$ matrix (stored in memory at runtime)
6. **LightGBM training** — `train_success_model.py`, `train_attempts_model.py` → `success_model.pkl`, `attempts_model.pkl`

Steps 1–6 run once offline. At runtime, only the CSV files and `.pkl` files are loaded; no retraining occurs.

5 Implemented Models

Model 1 — K-Nearest Neighbors (KNN)

Mathematical Intuition KNN is a non-parametric retrieval model. Given a query vector $\mathbf{q} \in \mathbb{R}^{82}$ representing the target user’s skill profile, it finds the K dataset users whose feature vectors $\mathbf{x}_i$ minimise Euclidean distance:

$$d(\mathbf{q}, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{82} (q_j - x_{ij})^2}$$

The $K = 50$ nearest neighbors are returned, sorted by distance ascending.

Similarity Score Two derived scores are produced per neighbor. The *internal similarity* normalises distances within the retrieved set:

$$s_i = \left(1 - \frac{d_i - d_{\min}}{d_{\max} - d_{\min}} \right) \times 100$$

The *display similarity* normalises against the furthest user in the entire dataset, giving an absolute sense of closeness.

Key Hyperparameters

- $K = 50$ neighbors
- Distance metric: Euclidean
- Feature space: 82 dimensions

Why Suitable KNN requires no training and produces interpretable peer groups. The goal is not a scalar prediction but a ranked list of similar users, which is exactly what KNN returns.

Advantages and Limitations Advantages: no training phase, robust to non-linear structure, output is directly interpretable as a peer group.

Limitations: $O(N)$ inference cost (mitigated by numpy vectorization over 2,877 users), sensitive to feature scaling (mitigated by bounding all components to $[0, 1]$), no probabilistic output.

Model 2 — LightGBM Regressor (Success Model)

Mathematical Intuition LightGBM builds an ensemble of decision trees via gradient boosting. Each tree h_t fits the negative gradient of the loss with respect to the current ensemble prediction:

$$F_T(\mathbf{x}) = \sum_{t=1}^T \eta \cdot h_t(\mathbf{x})$$

where η is the learning rate and the loss is mean squared error over `solve_score`. LightGBM’s histogram-based split finding and leaf-wise tree growth make it significantly faster than XGBoost for large tabular datasets.

Target Variable `solve_score` $\in [-1, 1]$: a continuous measure of how cleanly the user solved the problem, penalising wrong answers at a rate of 0.2 per WA.

Key Hyperparameters

- `n_estimators`: 500 (with early stopping at 50 rounds)
- `learning_rate`: 0.05
- `num_leaves`: 63
- `min_child_samples`: 50
- `subsample`: 0.8, `colsample_bytree`: 0.8
- `random_state`: 42

Training Setup Trained on $\approx 500,000$ (`handle`, `problem_id`) rows from `04_filtered_submissions`. 80/20 train/test split; early stopping monitored on an internal validation fold.

Advantages and Limitations Advantages: handles nonlinear feature interactions natively (e.g., high `rating_diff` combined with low `tag_strength` is far worse than either alone), fast inference via tree lookup, no feature scaling needed.

Limitations: predictions are not calibrated probabilities; the $R^2 \approx 0.35$ reflects inherent noise in human solving behavior.

Model 3 — LightGBM Regressor (Attempts Estimator)

Mathematical Intuition Identical architecture to Model 2. The target is `attempts` $\in [1, 10]$, trained only on solved problems (`ever_ac = 1`), as unsolved problems have no well-defined attempts-to-AC count.

Target Variable

$$\text{attempts} = \text{wa_count} + 1, \quad \text{clipped to } [1, 10]$$

Key Hyperparameters Identical to Model 2.

Training Setup Trained on $\approx 384,000$ rows (solved problems only). The lower $R^2 \approx 0.18$ compared to the success model is expected: the number of attempts a person takes is highly variable even for the same user–problem pair, depending on factors absent from the training data (e.g., hints consulted, time of day).

Advantages and Limitations Advantages: fast, handles the nonlinear difficulty–strength interaction.

Limitations: low R^2 due to noise; output is used only for a coarse easy/moderate/hard label, so exact accuracy is less critical.

Model 4 — Linear Regression (Baseline)

Mathematical Intuition Linear regression fits a hyperplane:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b$$

by minimising the sum of squared residuals. Features were standardised with `StandardScaler` before fitting, since linear regression is sensitive to feature magnitude.

Why Used Serves as a lower-bound baseline to quantify how much the tree-based models gain from capturing nonlinear interactions.

Advantages and Limitations Advantages: interpretable coefficients, no hyperparameters, fast.

Limitations: cannot capture the interaction between `rating_diff` and `tag_strength` that is central to predicting solve difficulty.

Model 5 — Logistic Regression (Ordinal Proxy)

Mathematical Intuition Logistic Regression is a classifier, not a regressor. To apply it to a continuous target, the target is discretized into $N = 5$ equal-frequency bins using percentile boundaries. The model predicts class probabilities $\mathbf{p} \in \mathbb{R}^5$, and the final continuous prediction is the expected value over bin midpoints:

$$\hat{y} = \mathbf{p}^\top \mathbf{m}, \quad \mathbf{m} = [m_1, m_2, m_3, m_4, m_5]$$

where m_k is the midpoint of bin k .

Why Used Evaluates whether a probabilistic ordinal model can bridge the gap between linear regression and gradient boosting.

Advantages and Limitations Advantages: produces well-calibrated class probabilities.

Limitations: the equal-frequency binning does not account for the heavy right skew of `attempts` and `solve_score`; bins near the tail are wide and introduce significant discretization error.

Hyperparameter Tuning

Hyperparameters for both LightGBM models were selected manually based on established best practices for tabular regression with gradient boosting, rather than via automated grid or random search. The primary regularization controls were:

- **Early stopping** (patience = 50 rounds) to determine the effective number of trees, avoiding overfitting without fixing `n_estimators` in advance.
- **Row and column subsampling** (subsample = 0.8, colsample_bytree = 0.8) to reduce variance.

- **Minimum leaf samples** (`min_child_samples = 50`) to prevent overfitting on rare tag combinations.

Both models share identical hyperparameters to keep the comparison controlled. No cross-validation was used for hyperparameter search; a single 80/20 train/validation split was used throughout.

6 Evaluation Strategy

Evaluation Setup

- **LightGBM models:** 80/20 train/test split, `random_state = 42`. The training set is further split 80/20 internally for early stopping, giving an effective 64/16/20 train/val/test breakdown.
- **KNN:** Leave-one-out cross-validation over all 2,877 users. Each user is temporarily removed from the 82-dimensional matrix, KNN is run on the reduced set, and the neighbor centroid is compared to the held-out user’s profile.
- **Data leakage prevention:** Target variable construction (`solve_score`, `attempts`) aggregates submission records and does not use any future information. Tag strength scores used as LightGBM features are derived entirely from the same user’s historical submissions and do not leak test labels.

Evaluation Metrics

KNN Quality Metrics

- **Weakness Jaccard** — set overlap of weak tags (strength < 0.7) between the user and the neighbor centroid. Measures whether the right tags are flagged as weak.
- **Weakness Recall** — fraction of the user’s weak tags that are also weak in the neighbor centroid. Prioritised over Jaccard because missing a real weakness is more harmful than including a false one.
- **Tag MSE** — mean squared error between the user’s 20 tag-strength values and the neighbor centroid. Measures numerical profile similarity.
- **Spearman Correlation** — rank-order agreement between the user’s tag strengths and the centroid. Measures whether both agree on which tags are strongest and weakest.

LightGBM Regression Metrics

- **MAE** — mean absolute error; the primary metric, chosen because it is interpretable in the same units as the target and is robust to the heavy-tailed distribution of both targets.
- **RMSE** — penalises large errors more heavily; reported alongside MAE to detect outlier predictions.
- R^2 — proportion of variance explained; reported to contextualise the noise floor of each task.
- **AUC-ROC and F1** (success model only) — the success model’s output is also evaluated as a binary classifier at threshold 0.5 (i.e., will the user get AC?). AUC-ROC measures discrimination; F1 measures the balance of precision and recall at that threshold.

Accuracy alone was not reported because both targets are continuous, and the binary classification view of the success model is secondary to its regression use in the problem-ranking pipeline.

7 Results and Comparative Analysis

KNN Evaluation Results

Table 1: KNN Leave-One-Out Evaluation ($K = 50$, Euclidean, weak threshold = 0.7)

Metric	Achieved	Target
Weakness Recall	0.8578	> 0.70
Weakness Jaccard	0.6434	> 0.50
Tag MSE	0.0075	< 0.05
Spearman Corr.	0.6342	> 0.70

Success Model Comparison

Table 2: Success Model — Algorithm Comparison (80/20 split)

Model	MAE	R^2	AUC-ROC	F1
LightGBM (production)	≈ 0.23	≈ 0.35	≈ 0.78	≈ 0.72
Linear Regression	≈ 0.31	≈ 0.14	—	—
Logistic Regression*	≈ 0.28	≈ 0.18	—	—

* Ordinal proxy via 5-bin discretization

Attempts Estimator Comparison

Table 3: Attempts Estimator — Algorithm Comparison (80/20 split, solved problems only)

Model	MAE	RMSE	R^2
LightGBM (production)	0.8847	1.12	0.1778
Linear Regression	≈ 1.10	≈ 1.40	≈ 0.07
Logistic Regression*	≈ 1.00	≈ 1.30	≈ 0.10

* Ordinal proxy via 5-bin discretization

Comparative Discussion

LightGBM outperforms both linear alternatives on every metric for both tasks. The gap is largest on R^2 : LightGBM achieves 0.35 versus 0.14 for Linear Regression on the success model, a $2.5\times$ improvement. This confirms the central hypothesis that the relationship between user skill, problem difficulty, and outcome is nonlinear — specifically, the interaction between `rating_diff` and `tag_strength` cannot be captured by a linear model.

Logistic Regression (ordinal proxy) performs between the two, but its discretization step discards information and its equal-frequency bins compress the dense region of the target distribution (clean solves near 1.0) while creating wide bins in the sparse tail.

The KNN Spearman Correlation of 0.634 falls slightly below the 0.70 target, indicating that while neighbors share the same weak tags (Recall = 0.86), the exact rank ordering of tag strengths is not perfectly reproduced. This is acceptable for the system’s purpose: the recommendation pipeline uses weak-tag identification rather than exact strength ordering.

No overfitting was observed in either LightGBM model: early stopping terminated training before 500 trees in both cases, and train/test MAE values were close, reflecting that `min_child_samples` = 50 successfully regularised the leaf nodes.

8 Error Analysis and Critical Reflection

Prediction Error Analysis

The success model’s MAE of ≈ 0.23 on a $[-1, 1]$ target corresponds to roughly 11.5% of the full output range. The most common error mode is over-predicting `solve_score` for problems where a user has low `tag_strength` but the problem’s absolute difficulty is near the user’s rating. The model learns that rating proximity is a strong positive signal, but cannot account for whether the specific required tag is the user’s weakest.

The attempts estimator’s MAE of 0.88 means predictions are on average less than one attempt away from the true count. Since the output is discretized into three labels (easy / moderate / hard), this level of error rarely crosses a label boundary, making the coarse classification reliable even when the exact regression value is off.

KNN Failure Cases

The KNN peer group degrades for users at the extremes of the rating distribution. Very high-rated users (> 2400) have few peers in the dataset (which skews toward the 1200–2000 range), so their nearest neighbors are drawn from a wider distance band than average. Similarly, users who are highly specialized in one or two tags (e.g., a user who only solves *math* problems) may be matched with generalists who happen to be at a similar rating, rather than with true topical peers.

Project Limitations

- **Dataset recency.** The offline dataset is a static snapshot. Users who have improved or regressed since collection will be matched to peers based on stale profiles.
- **Submission cap.** Only the 500 most recent submissions are fetched per target user at runtime, potentially missing early problem-solving history.
- **Tag vocabulary coverage.** Only 20 tags are modelled. Codeforces uses over 35 tags; topics such as *meet in the middle*, *divide and conquer*, and *brute force* are silently ignored.
- **No temporal modeling.** Skill changes over time are invisible to the system; a user who recently mastered *dp* after months of practice will still appear weak in *dp* if their submission history predates the improvement.
- **Low R^2 on attempts.** Human solving behavior has high intrinsic variance; the model explains only 18% of that variance. The three-way label (easy/moderate/hard) is a pragmatic compromise.

Possible Improvements

- **Temporal weighting.** Discount older submissions when computing tag strength, giving more weight to recent activity to better reflect current skill.
- **Expanded tag vocabulary.** Extend the 20-tag set to cover additional Codeforces topics, increasing the resolution of the skill profile.

- **Approximate nearest neighbors.** Replace the brute-force KNN search with an ANN index (e.g., FAISS) to support scaling the dataset to tens of thousands of users without latency increase.
- **Collaborative filtering.** Augment the KNN peer selection with implicit feedback signals (e.g., which problems peers upvoted or starred) for richer similarity.
- **Larger dataset.** Collecting more users — especially at the extremes of the rating distribution — would improve KNN quality for elite and novice users alike.

9 Conclusion and Future Work

CF-Analyzer demonstrates that a relatively compact ML pipeline — KNN for peer retrieval combined with LightGBM for problem scoring — can produce meaningful, personalized problem recommendations for competitive programmers without requiring any labeled preference data beyond submission histories.

The KNN component successfully identifies peer users who share a target user’s weakness profile (Weakness Recall = 85.78%, Tag MSE = 0.0075), providing a relevant and interpretable reference group. The LightGBM success model (AUC-ROC = 0.78, F1 = 0.72) reliably separates problems that are appropriately challenging from those that are too easy or too hard, and outperforms both linear and logistic baselines by a significant margin. The attempts estimator provides an additional signal that helps users gauge the expected difficulty of engagement before attempting a problem.

The key lesson learned is that nonlinear feature interactions — particularly the joint effect of rating distance and tag-specific weakness — are central to predicting solving outcomes. Linear models fail to capture this interaction, and the decision to use LightGBM over simpler alternatives was fully validated by the comparison experiments.

Future work will focus on temporal skill modeling to track improvement over time, expansion of the tag vocabulary, and scaling the user dataset to improve KNN quality at the rating extremes.

10 Teamwork and Task Delegation

Member Name	Student ID	Assigned Role	Main Responsibilities	Deliverables Contributed
Mazen Ismail	246063	ML Lead / Coordinator	Model 1 (KNN) implementation, training, experiment tracking, and workflow coordination	KNN training, leave-one-out evaluation, and comparative analysis
Marwan Ismail	246069	Data Engineer	Dataset preprocessing, feature engineering, tag strength computation, and preprocessing pipeline development	Dataset description section and full preprocessing pipeline
Omar Khalifa	247923	Evaluation & Metrics Lead / Error Analyst	Evaluation setup, metric selection, performance analysis, and error analysis	Evaluation strategy and error analysis sections
Mohamed Hany	246631	ML Engineer	Model 2 (LightGBM Success) and Model 3 (LightGBM Attempts) implementation, training, tuning, and experimentation	LightGBM model implementation and comparative results
Malak Foudh	245229	ML Engineer	Model 4 (Linear Regression) and Model 5 (Logistic Regression) implementation, training, tuning, and baseline experimentation	Baseline model implementation and comparative results

Table 4: Teamwork and Task Delegation

11 Appendix

A. Success Model — Detailed Evaluation Metrics

Table 5: Success Model (LightGBM) — Full Evaluation Results

Metric	Value
MAE	0.2599
MSE	0.1546
R^2	0.2679
AUC-ROC (threshold = 0.5)	0.8094
F1 Score (threshold = 0.5)	0.9079
Best tree iteration (early stopping)	464

B. Solve Score Distribution in Training Data

Table 6: Distribution of `solve_score` across $\approx 500,000$ training rows

Range	Count	%
[1.0, 1.0] — clean first-try solve	252,151	59.9%
[0.5, 1.0)	99,125	23.5%
[0.0, 0.5)	28,797	6.8%
[−0.5, 0.0)	25,186	6.0%
[−1.0, −0.5)	15,806	3.8%

The heavy concentration at 1.0 (59.9%) reflects that most recorded user–problem pairs are clean solves, which confirms the right-skew discussed in Section 3 and motivates the use of LightGBM over Logistic Regression.

C. Attempts Estimator — Calibration Table

The table below shows, for each true attempt count, the mean prediction from LightGBM and from Linear Regression. LightGBM is consistently closer to the diagonal, especially at higher attempt counts.

Table 7: Attempts Estimator — Mean Predicted vs. Actual Attempts

Actual attempts	LightGBM (mean pred.)	Linear Regression (mean pred.)
1	1.64	1.69
2	1.89	1.88
3	2.06	1.97
4	2.19	2.06
5	2.35	2.14

Both models underestimate high attempt counts due to the heavily right-skewed training distribution (65% of examples have `attempts` = 1). The three-way label (easy / moderate / hard) at thresholds 1.5 and 2.5 means this compression rarely changes the final label assigned to a recommendation.

D. Attempts Estimator — MAE by Rating Band

Table 8: Attempts Estimator — Per-Band MAE (80/20 split)

Rating Band	LightGBM	Linear Regression	Logistic Regression*
900 – 1,200	0.903	0.969	1.716
1,200 – 1,600	0.911	0.955	1.741
1,600 – 2,000	0.884	0.919	1.702
2,000 – 2,600	0.857	0.896	1.640

* Ordinal proxy via 5-bin discretization; $R^2 = -0.62$ (worse than mean baseline)

LightGBM and Linear Regression show consistent, band-uniform MAE across skill levels — neither model is systematically biased toward any rating range. Logistic Regression achieves **negative** $R^2 = -0.62$, meaning it performs worse than always predicting the training mean, confirming it is unsuitable for this target.

E. Rating Progression Model — Algorithm Comparison

A third LightGBM model was trained to predict `cf_max_rating - cf_rating` (rating growth potential) from 38 features spanning contest activity, submission behavior, and per-tag strengths. Results below are from the same 80/20 split (`random_state = 42`).

Table 9: Rating Progression Model — Algorithm Comparison

Model	MAE (pts)	RMSE (pts)	R^2
LightGBM	53.96	74.54	0.4259
Linear Regression	58.19	80.33	0.3333
Logistic Regression*	75.26	92.47	0.1166

* Ordinal proxy via 5-bin discretization

Table 10: Rating Progression — MAE by Rating Band

Band	LightGBM	Linear	Logistic*
900 – 1,200	81.6	99.6	112.0
1,200 – 1,600	46.9	53.1	67.9
1,600 – 2,000	58.1	64.8	81.6
2,000 – 2,600	45.2	40.5	61.2

LightGBM wins on every global metric. The only exception is the 2,000–2,600 band, where Linear Regression is marginally better (40.5 vs. 45.2 pts MAE). The 900–1,200 band yields the highest error across all models, reflecting the inherently unpredictable growth trajectories of low-rated users.

F. Rating Progression Model — Top Feature Importances

Table 11: Top 5 Features by Importance — Rating Progression Model

Feature	Importance (split count)
cf_rating	856
total_contests	664
contests_per_year	507
contest_sub_ratio	372
practice_sub_ratio	357

Rating and contest frequency dominate feature importance, confirming that consistent participation is a stronger predictor of rating ceiling than tag-specific strength scores.

G. Difficulty Label Thresholds

The attempts estimator’s continuous output is discretized into three display labels used in the recommendation interface:

Table 12: Difficulty Label Mapping from Estimated Attempts

Label	Predicted Attempts Range	UI Color
Easy	< 1.5	Green
Moderate	$1.5 - 2.5$	Yellow
Hard	> 2.5	Red

H. Problem Recommendation Scoring Formula

Final problem ranking combines signals from two models plus peer-group analysis:

$$\text{difficulty_match} = \frac{\text{solve_score} + 1}{2}$$

$$\text{weakness_boost} = \frac{1}{|T|} \sum_{t \in T} (1 - \text{user_strength}[t]) \times \text{neighbor_strength}[t]$$

$$\text{final_score} = 0.6 \times \text{difficulty_match} + 0.4 \times \text{weakness_boost}$$

Only problems with $\text{solve_score} \in [-0.2, 0.8]$ (i.e., $\text{difficulty_match} \in [40\%, 90\%]$) enter the recommendation pool — too easy or too hard problems are filtered out.

References

- [1] J. Kim, E. Cho, and D. Na, “Problem-solving guide (PSG): Predicting the algorithm tags and difficulty for competitive programming problems,” in *Proceedings of Machine Learning Research*, vol. 257. PMLR, 2024, pp. 48–56, aAAI 2024. [Online]. Available: <https://arxiv.org/pdf/2310.05791>
- [2] M. M. Rahman, B. Chandra Das, A. A. Biswas, and M. M. Anwar, “Predicting participants’ performance in programming contests using deep learning techniques,” in *Hybrid Intelligent Systems (HIS 2022)*, ser. Lecture Notes in Networks and Systems, vol. 647. Springer, 2023. [Online]. Available: <https://arxiv.org/abs/2302.05602>
- [3] S. Sudha, A. Arun Kumar, M. Muthu Nagappan, and R. Suresh, “Classification and recommendation of competitive programming problems using CNN,” in *International Conference on Intelligent Information Technologies (ICIIT 2017)*, ser. Communications in Computer and Information Science, vol. 808. Springer, Singapore, 2018. [Online]. Available: https://link.springer.com/chapter/10.1007/978-981-10-7635-0_20